

Control Statements

Amir Jamalian



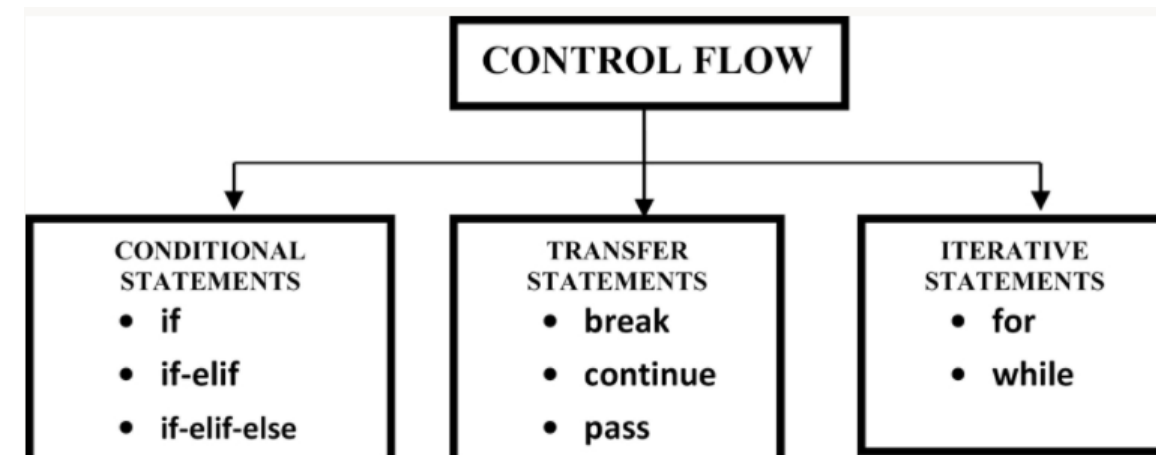
Gisma
University
of Applied
Sciences





Control Statements

- The code is always executed from the first to the last line of the program
 - Known as sequential control flow
- A control statement identifies the controlled circulation of set-of-instructions in which they will execute
 - There are three basic methods of control movement in it





Conditional Statements

- Conditional statements are used to control the flow of execution within a program
 - Based on some condition
- These conditions represent some choice point that will be evaluated to True or False
 - if $x > 10$



If Statement

- If the condition is True, then we will execute the indented statement
 - The condition must evaluate to True or False
 - Or an equivalent value
- The indentation is very important in Python
 - The layout of your program is part of your program
 - Unlike language such as Java and C#
 - All lines indented to the same level after the if statement will automatically be part of the if statement
 - The : character is key in separating out the conditional part of the if statement from the statements

```
if <condition>:  
    statement
```

```
if x > 0:  
    → print("x is positive!")
```



Comparison Operators

- Comparison operators return Boolean values

Operator	Name	Example
==	Equal	<code>x == y</code>
!=	Not equal	<code>x != y</code>
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>



Logical Operators

- Logical operators can be used to combined Boolean expressions together
 - To create more complex conditions

Operator	Description	Example
and	Returns True if both statements are true	<code>x > 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x > 10</code>
not	Reverse the result, returns False if the result is true	<code>not(x > 5 and x < 10)</code>



Else in an If Statement

- Else is an optional element

- That can be run if the conditional part of the if statement returns False

```
if x > 0:  
    print("x is positive!")  
else:  
    print("x is not positive!")
```



Elif in an If Statement

- We may have several conditions to test

- This else-if scenario is supported in Python by the elif element of an if statement

```
if x > 0:  
    print("x is positive!")  
elif x < 0:  
    print("x is negative!")  
else:  
    print("x is zero!")
```




Nesting If Statements

- It is possible to nest one if statement inside another
 - The indentation is key here

```
if x >= 0:
    if x > 0:
        print("x is positive!")
    else:
        print("x is zero!")
else:
    print("x is negative!")
```



If Expressions

- Recap: expressions and statements are different
 - Expressions return a value
 - Statements do not
- If expression is a short hand form of an if statement that returns a value
 - It is quite common to want to assign a specific value to a variable dependent on some condition

```
<result1> if <condition-is-met> else <result2>
```

```
status = "positive" if x > 0 else "not positive"
```





Iterative Statements

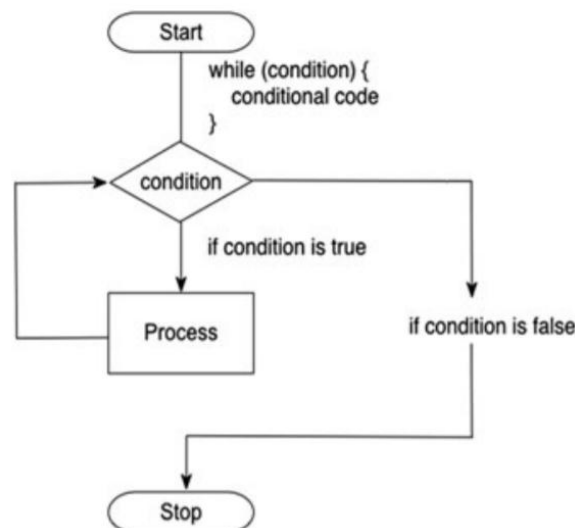
- The iterative statements are used to control the repeated execution of selected statements
 - While
 - For



While Loop

- The while loop iterates (or repeats) one or more code statements as long as the test condition (expression) is True

- Suitable for when the number of repetition is not known
- Testing the condition is performed before each iteration



```
while <test-condition-is-true>:  
    statement or statements
```

```
x = [1, 2, 3]  
while x:  
    print(x)  
    x.pop(0)
```

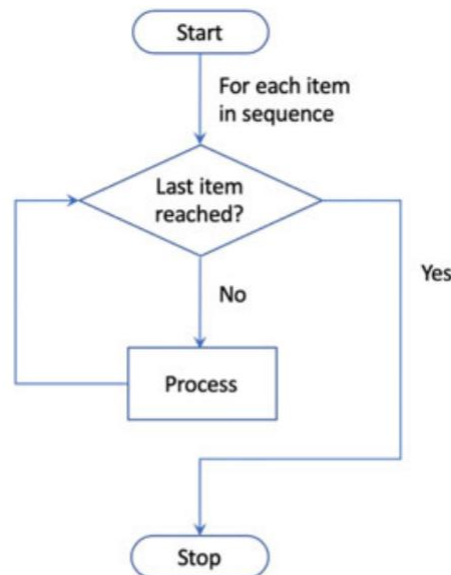


For Loop

[Hunt 2019]

13

- The for loop is a more concise and also clearer iteration statement when we know how many times we want to iterate
 - It step a variable through a series of values until a given test is met



```
for <variable-name> in <a-collection>:  
    statement(s)
```

```
for i in [1, 2, 3]:  
    print(i)
```



Range Function

- The range(start, stop, step) function returns a sequence of numbers
 - Starting from 0 (by default)
 - Stops before a specified number
 - Increments by 1 (by default)
- range(10) is the same as range(0, 10, 1)
 - [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
- Variable names should be meaningful in general
 - However, it is common to define loop variable names as i, j, etc.
 - Consider using these variable names in looping constructs
 - Avoid using them elsewhere

```
for i in range(10):  
    print(i)
```

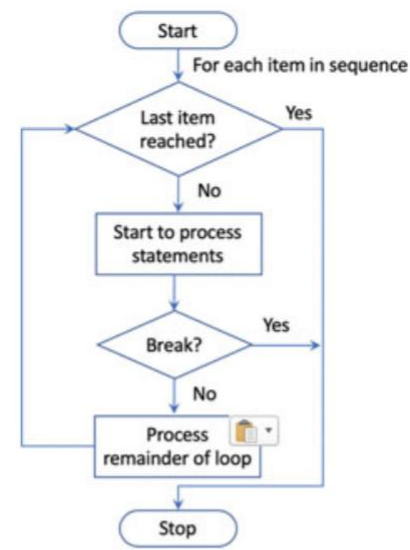


Break Loop Statement

[Hunt 2019]

15

- The break statement allows a developer to alter the normal cycle of the loop based on some criteria which may not be predictable in advance
 - Typically, a guard (if) statement is placed on the break
 - So that the break statement is conditionally applied when appropriate



```
for i in range(10):  
    print(i)  
    if i == 5:  
        break
```

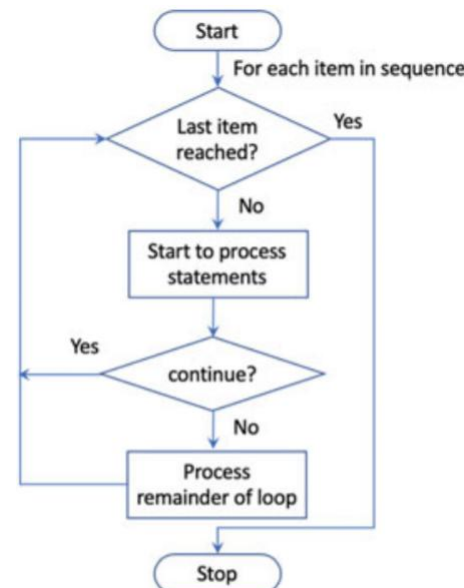


Continue Loop Statement

[Hunt 2019]

16

- The continue statement allows to skip over part of a loop's iteration for a particular step value
 - But then to continue with the remaining step values in the sequence
 - A guard (if statement) can be used to determine when the continue statement should be executed



```
for i in range(10):  
    if i == 5:  
        continue  
    print(i)
```




Pass Statement

- If a block is needed that will not execute anything then, you can declare the empty block using a pass keyword

```
for i in range(10):  
    if i % 2 == 0:  
        print(i)  
    else:  
        pass
```